Computer Arithmetic & Numerical Accuracy

IEEE 754, Precision, Catastrophic Cancellation

9th of April 2026

Jimmy Jessen Nielsen
jjn@es.aau.dk

Dept. of Electronic Systems
Aalborg University

Computer Arithmetic & Numerical Accuracy

Today's Agenda

- **Peer Review MP2** (∼60 min)
  - Pairs — one aspect at a time; both show briefly, then compare and discuss
- **Break** (10 min)
- **Theory**: Computer arithmetic (30 min)
  - Integer representation, IEEE 754, precision limits
  - Famous disasters, catastrophic cancellation, Horner's method
  - Condition number — with a Mandelbrot connection
- **Studio**: Exercises and MP3 milestones — MP3 opens today
  - E1: Machine epsilon E2: Catastrophic cancellation
  - MP3 M1: Trajectory divergence MP3 M2: Sensitivity map

By the end of today: peer review done, float32/float64 behaviour explored in theory and in code, MP3 milestones M1–M2 recorded.

Physical Peer Review MP2

Peer Review MP2

Instructions

Purpose: Review a fellow student's MP2 work aspect by aspect — compare approaches, discuss what worked.

Format: Interleaved aspect review

Structure:

1. **Setup** (5 min): Pairing and seating; open code and performance notebook
2. **Aspect rounds** (5–10 min each $\times$ 5 aspects): A shows briefly $\rightarrow$ B shows briefly $\rightarrow$ compare and discuss
3. **Reflection** (5 min): Write down two takeaways

Discussion prompts for each aspect:

- What did each person do differently?
- Strengths and weaknesses of each approach?
- What would you keep — and what would you change?

Peer Review MP2

Pair Formation

Pick up a post-it note from the front — colour = your education

- Circulate the room and find someone with a *different* colour
- Sit next to each other and open your code on screen

Goal: pair across educations — different backgrounds $\Rightarrow$ more to learn

Today's format: aspect-by-aspect review

- Both show their approach to each aspect — then compare and discuss
- No need to present the whole project from start to finish

Remember: Constructive feedback, not criticism!

Peer Review MP2

Aspect-by-Aspect Guide (40 min total)

For each aspect: A shows briefly → B shows briefly → compare and discuss.

1. **Multiprocessing** (∼8 min) — chunk sweep and speedup
   - What chunk size did you converge on? How does the speedup compare?
2. **Dask local** (∼8 min) — LocalCluster performance, scheduling overhead
   - How visible was the overhead at small resolutions? At $\geq$4096?
3. **Dask cluster** (∼10 min) — Strato setup, worker scaling table
   - How did you set up the cluster? Any surprises in scaling?
4. **Performance table** (∼7 min) — all six rows, Numba baselines
   - Are speedups fair comparisons? Same resolution and hardware for each pair?
5. **Code quality + Git history** (∼7 min) — read one function each
   - Is it clear what the code does? Does the Git log tell the story of the work?

Peer Review MP2

Individual Reflection (5 min)

Write down your answers:

1. What was the most interesting difference between your partner's Dask implementation and yours?
2. How did your cluster results compare to what you expected, and to your Dask local results?
3. What would you do differently in your code based on what you saw?
4. One technique or habit from today that you will carry into MP3?

Peer Review MP2

Class Discussion (5 min)

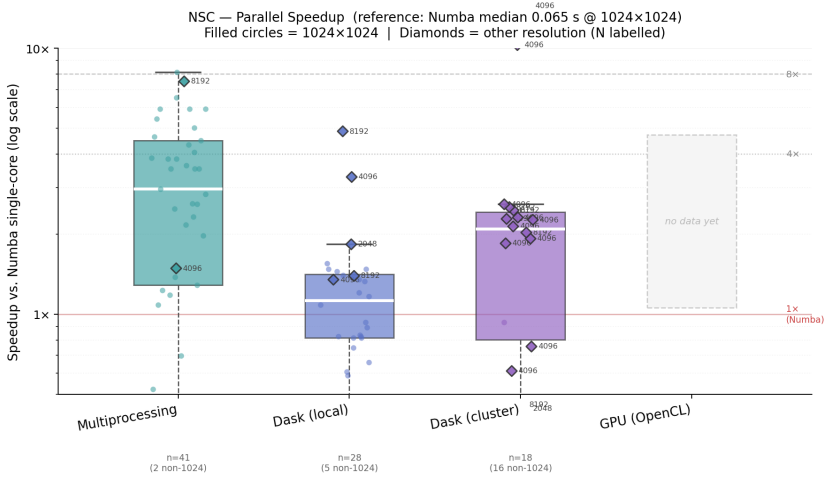
Share insights with the whole class — volunteers welcome:

- What was the most surprising result from the cluster benchmarks?
- Did anyone achieve near-linear speedup with worker count? What helped?
- What was the hardest part of the Strato setup?
- Any creative optimisation or code approach that others should know about?

Computer Arithmetic & Numerical Accuracy

Computer Arithmetic & Numerical Accuracy

Performance Tracker



Computer Arithmetic & Numerical Accuracy

Integer Representation

Why integers first? Floating-point is built on top of integer bit patterns.

Two's complement (8-bit):

Bits	Unsigned	Signed
00000000	0	0
01111111	127	127
10000000	128	−128
11111111	255	−1

Overflow wraps silently:

$127 + 1 = -128$ in `int8`

— no exception, no warning.

The CPU has no way to detect this. Range checks must be added explicitly.

Python / NumPy integer types:

Type	Range
`np.int8`	$-128 \ldots 127$
`np.uint8`	$0 \ldots 255$
`np.int16`	$-32768 \ldots 32767$ (Ariane 5)
`np.uint16`	$0 \ldots 65535$
`np.int32`	$\approx \pm 2 \times 10^9$
`np.int64`	$\approx \pm 9 \times 10^{18}$
`int`	unbounded (Python native)

`uint32`/`uint64`: same ranges but non-negative.

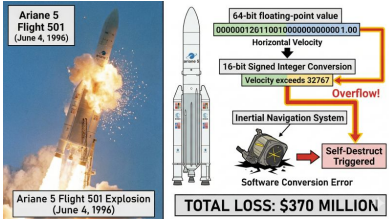
NumPy default integer: `int64` on 64-bit systems.

Computer Arithmetic & Numerical Accuracy

Ariane 5 (1996) — Integer Overflow

- Flight 501 reused inertial navigation software from Ariane 4
- Code converted horizontal velocity: 64-bit float → 16-bit integer
- Ariane 5 flew faster: velocity exceeded `int16` range ($> 32\,767$)
- Ada raises an exception on overflow (unlike C/NumPy, which wrap silently)
- Exception uncaught; handler wrote diagnostic data to flight bus; computers misread it as flight data → both shut down → self-destruction after 37 s
- $370 million lost; 10 years of development gone in 39 seconds

Root cause: no range check; software reused from a slower rocket without re-validation

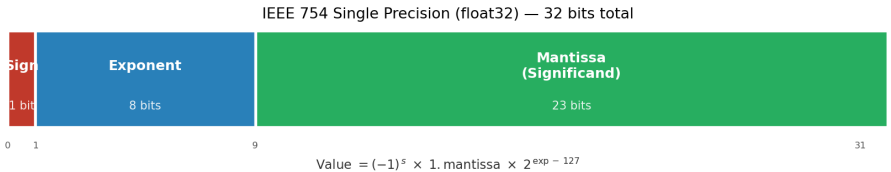


Computer Arithmetic & Numerical Accuracy

IEEE 754 Floating-Point

Why not just use integers? Need to represent very small and very large numbers.

IEEE 754 float32 — 32 bits: $(-1)^{\text{sign}} \times 1.\text{mantissa} \times 2^{\text{exponent}-127}$



- **Sign** (1 bit): 0 = positive, 1 = negative
- **Exponent** (8 bits): stored as $e + 127$ (bias); actual exponent $= e - 127$; range -126 to $+127$
- **Mantissa** (23 bits): fractional bits after the implicit leading `1.` (normalised form)

Computer Arithmetic & Numerical Accuracy

IEEE 754 — Decimal to Binary: 13.625

Example: *Encode* 13.625 *as float32.*
Standard approach: Convert integer and fractional parts separately.

Integer part: 13
(*repeated division by 2*)

Quotient	Remainder
$13 \div 2 = 6$	**1**
$6 \div 2 = 3$	**0**
$3 \div 2 = 1$	**1**
$1 \div 2 = 0$	**1** ← read first

$\Rightarrow 1101_2$ (read column upwards)

Fractional part: 0.625
(*repeated multiplication by 2*)

$\times 2$	Integer part	Remainder
$0.625 \times 2 = 1.25$	**1** ← read first	0.25 ↓
$0.25 \times 2 = 0.5$	**0**	0.5 ↓
$0.5 \times 2 = 1.0$	**1**	0 (done)

$\Rightarrow .101_2$ (read column downwards)

Combined: $1101.101_2 = 1.101101 \times 2^3$ (normalise: shift point 3 left)

IEEE 754 float32 encoding: $(-1)^{\text{sign}} \times 1.\text{mantissa} \times 2^{\text{exponent}-127}$

- sign:0
- exponent: $3 + 127 = 130 = 10000010_2$
- mantissa: 10110100000000000000000 (the 101101 from normalised form, zero-padded up to 23 bits)

Computer Arithmetic & Numerical Accuracy

Precision Limits

Type	Bits	Significant digits	Machine ε
`float16`	16	≈ 3	$\approx 9.8 \times 10^{-4}$
`float32`	32	≈ 7	$\approx 1.2 \times 10^{-7}$
`float64`	64	≈ 15	$\approx 2.2 \times 10^{-16}$

Machine epsilon ε: the smallest number such that $1.0 + \varepsilon \neq 1.0$

- Also the maximum relative rounding error in a single operation
- In Python: `import numpy as np; np.finfo(np.float32).eps`

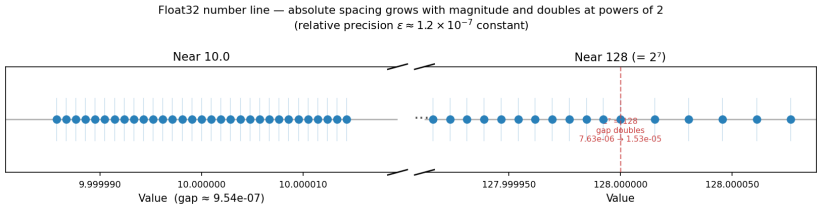
Significant digits: how many decimal digits can be trusted after a single operation.

- Operations that combine numbers of very different magnitudes can lose digits rapidly.

Computer Arithmetic & Numerical Accuracy

The Spacing Problem

Floating-point numbers are **not evenly spaced** on the number line.

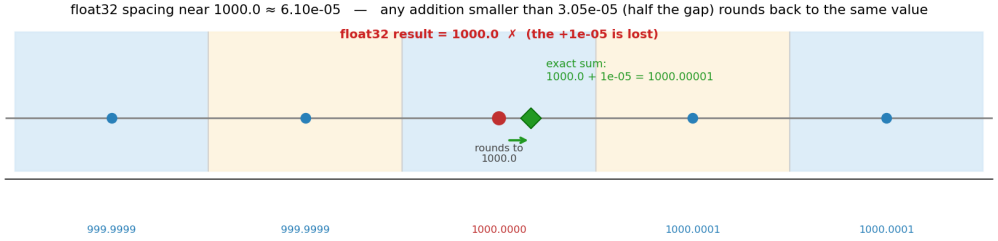


- **Absolute** spacing grows with magnitude; **relative** spacing is bounded by ε (between $\varepsilon/2$ and ε)
- Spacing is **piecewise constant** within each *binade* $[2^n, 2^{n+1})$ — right panel shows the jump at $2^7 = 128$: spacing doubles from 7.6×10^{-6} to 1.5×10^{-5}
- Adding numbers of vastly different magnitudes can lose the smaller one entirely

Computer Arithmetic & Numerical Accuracy

Magnitude Absorption

Adding a number smaller than **half the local spacing** leaves the result unchanged — the small value is absorbed entirely.



Each shaded zone shows which float32 value the interval rounds to. The green ♦ is the exact sum $1000.0 + 10^{-5} = 1000.00001$ — still inside the 1000.0 zone.

```
np.float32(1000.0) + np.float32(1e-5)    # => 1000.0  (unchanged)
np.float32(1000.0) + np.float32(4e-4)    # => 1000.0006  (rounds to next float32)
```

Computer Arithmetic & Numerical Accuracy

Special Values

IEEE 754 reserves bit patterns for special cases:

Value	How it arises	Propagates?
$+\infty$	positive overflow; `x / 0.0` ($x > 0$, NumPy)	yes
$-\infty$	negative overflow; `x / 0.0` ($x < 0$, NumPy)	yes
NaN	`0.0 / 0.0`, $\infty - \infty$, `sqrt(-1)`	yes
± 0	underflow or `-0.0` literal	context-dependent

Plain Python `float` raises `ZeroDivisionError` on `x / 0.0`; NumPy returns `inf`/`-inf` with a `RuntimeWarning`.

NaN propagates silently: any arithmetic with NaN produces NaN. Comparisons with NaN always return False (including `NaN == NaN`).

- Check for NaN: `np.isnan(x)`
- Check for finite: `np.isfinite(x)`
- Silent NaN propagation is a common source of hard-to-debug errors

Computer Arithmetic & Numerical Accuracy

Why It Matters — Disasters

Classic examples

- **Patriot Missile (1991)**
  0.1 s in fixed-point; 100 h accumulation ⇒ 0.34 s offset. Scud missed; 28 killed.
- **Sleipner A (1991)**
  FEM underestimated shear by 47 %. Platform sank; $700M lost.
- **Vancouver Exchange (1982)**
  Index truncated (not rounded) per trade. After 22 months: 520 instead of 1100; jumped 11.5% on reset.
- **Ariane 5 (1996)**
  Velocity overflowed `int16`; exception flooded flight bus. $370M, 37 s.

More recent

- **Boeing 787 (2015)**
  32-bit counter at 10 ms ticks overflows after ≈248 days ⇒ potential loss of all AC power. FAA mandated periodic reboot.
  `theguardian.com` "Boeing 787 software bug"
- **COVID-19 UK (2020)**
  Excel `.xls` 16-bit row limit (65 536). ≈16 000 cases silently dropped.
  `theguardian.com` "How Excel may have caused loss of 16000 covid tests"
- **GPS rollover (2019)**
  10-bit week counter wraps every 1 024 weeks (≈19.7 yr). Older receivers reported navigation errors on 6 Apr 2019.
  `gps.gov` "GPS week number rollover"

Lesson: numerical accuracy is not a theoretical concern — and it keeps happening.

Computer Arithmetic & Numerical Accuracy

Catastrophic Cancellation

Goal: find both roots of $ax^2 + bx + c = 0$ using the quadratic formula:

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \qquad \text{(discriminant } \Delta = b^2 - 4ac\text{)}$$

When $b^2 \gg 4ac$, we have $\sqrt{\Delta} \approx |b|$, so *one* of the two roots requires computing $-b \mp \sqrt{\Delta} \approx |b| - |b|$ — subtracting two nearly-equal numbers. That is where all precision is lost.

Example: $x^2 - 10000.0001\,x + 1 = 0$ roots: $x_1 \approx 10000.0001$, $x_2 = 1/x_1 \approx 10^{-4}$

Step-by-step in float32 (7 significant digits):

- $b = -10000.0001$ rounds to $b_{32} = -10000.0$ (the .0001 exceeds float32 precision)
- $\Delta = b_{32}^2 - 4ac = 10^8 - 4.0$: spacing at 10^8 is 8 in float32, so 4.0 is absorbed $\Rightarrow \Delta_{32} = 10^8$
- $\sqrt{\Delta_{32}} = 10\,000.0$ exactly
- Small root (naive): $\dfrac{-b_{32} - \sqrt{\Delta_{32}}}{2a} = \dfrac{10\,000.0 - 10\,000.0}{2} = \mathbf{0}$

True $x_2 \approx 10^{-4}$; float32 naive result: **0 — 100% relative error**.

Computer Arithmetic & Numerical Accuracy

Catastrophic Cancellation: The Fix

Root cause: computing $-b \pm \sqrt{\Delta}$ when $|\sqrt{\Delta}| \approx |b|$ loses all significant bits in one of the two roots.

Fix: Vieta's formula — $x_1 \cdot x_2 = c/a$

1. Compute the **large** root first, where the $\pm$ avoids cancellation:
$$x_1 \leftarrow \frac{-b + \sqrt{\Delta}}{2a} = \frac{10\,000.0 + 10\,000.0}{2} = 10\,000.0 \quad \checkmark$$
2. Recover the small root using division (no subtraction of nearly-equal values):
$$x_2 \leftarrow \frac{c}{a \cdot x_1} = \frac{1}{10\,000.0} = 0.0001 \quad \checkmark$$

Relative error of Vieta's result: $\approx 10^{-8}$ (only float32 rounding); compare to 100% for the naive formula.

General principle: reformulate the algorithm to avoid subtracting nearly-equal numbers. More precision delays the problem but does not eliminate it.

Computer Arithmetic & Numerical Accuracy

Horner's Method — The Problem

Naive form: evaluate each term separately and sum:

$$p(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

This requires computing $x^2, x^3, \ldots, x^n$ — intermediate powers grow rapidly.

Example: $p(x) = 1200 - 150x + 11x^2 - 101x^3 + 12x^4$, $x = 10$, in `float16` (max $\approx 65\,504$):

Intermediate term	Exact value	float16 result
12×10^4	120 000	$+\infty$ (overflow!)
-101×10^3	$-101\,000$	$-\infty$ (overflow!)
$\infty + (-\infty)$	—	NaN
Final $p(10)$	19 800	NaN

The true answer 19 800 fits comfortably in float16. The individual terms do not.

Switching to float32 just delays the problem — for higher degree or larger x, the intermediate powers will overflow regardless.

Computer Arithmetic & Numerical Accuracy

Horner's Method — The Solution

Horner's form: factor out x at every step — no large intermediate powers:

$$p(x) = \big(\cdots\big((a_n \cdot x + a_{n-1}) \cdot x + a_{n-2}\big) \cdot x + \cdots + a_1\big) \cdot x + a_0$$

$$p(10) = \big(\big((12 \cdot 10 + (-101)\big) \cdot 10 + 11\big) \cdot 10 + (-150)\big) \cdot 10 + 1200$$

Same example in float16: all intermediates stay within range.

Step	Value	float16 OK?
$12 \times 10 + (-101)$	19	✓
$19 \times 10 + 11$	201	✓
$201 \times 10 + (-150)$	1 860	✓
$1\,860 \times 10 + 1\,200$	19 800	✓

- No large powers computed; intermediates bounded by the answer, not the terms
- Also requires fewer multiplications: n instead of $\sim n^2/2$

Take-away: algorithmic reformulation can eliminate precision problems that switching to a higher-precision type cannot fix.

Computer Arithmetic & Numerical Accuracy

Condition Number

Definition (general, p-norm form):

$$\kappa_p(f,\, x_0) \;=\; \lim_{\varepsilon \to 0^+} \sup_{\|x_\delta\|_p \le \varepsilon} \frac{\|f(x_0 + x_\delta) - f(x_0)\|_p \,/\, \|f(x_0)\|_p}{\|x_\delta\|_p \,/\, \|x_0\|_p}$$

Approximates relative change in output per relative change in input (exact in the limit $\delta \to 0$). A relative input error of ε produces a relative output error of $\approx \kappa\,\varepsilon$.

For a **differentiable scalar** f:

$$\kappa(f,\, x) \;=\; \lim_{\delta \to 0} \frac{|f(x+\delta) - f(x)|/|f(x)|}{|\delta|/|x|} = \lim_{\delta \to 0} \frac{|f(x+\delta) - f(x)|}{|\delta|} \cdot \frac{|x|}{|f(x)|} = |f'(x)| \cdot \frac{|x|}{|f(x)|} = \left| \frac{x\, f'(x)}{f(x)} \right|$$

Well-conditioned (κ small)

- Small input error $\to$ small output error
- Result reliable in finite precision
- $f(x) = x^2$:
  $\kappa = \left| \frac{x \cdot 2x}{x^2} \right| = 2$ (constant, all $x \neq 0$)

Ill-conditioned (κ large)

- Small input error $\to$ large output error
- More precision helps, but only up to a point
- $f(x) = x - a$ (a constant):
  $\kappa = \left| \frac{x}{x-a} \right| \to \infty$ as $x \to a$

Key insight: a large condition number is a property of the *problem*, not the algorithm. No numerical method can produce a reliable answer to a fundamentally ill-conditioned problem.

Computer Arithmetic & Numerical Accuracy

Condition Number — Example

Function: $f(x) = e^{kx}, \quad f'(x) = k\,e^{kx}$

Substituting into the scalar formula $\kappa(f,x) = \left|\dfrac{x\,f'(x)}{f(x)}\right|$:

$$\kappa(f,\,x) \;=\; \left|\frac{x\cdot k\,e^{kx}}{e^{kx}}\right| \;=\; |k\,x|$$

The e^{kx} terms cancel — κ depends only on k and the working point x_0.

Results for $k = 10$:

x_0	$\kappa = 10\,\lvert x_0\rvert$	
0.1	1	well-conditioned
1	10	
2	20	
5	50	← **see right**

At $k = 10$, $x_0 = 5$: $\quad \kappa = 50$

- A **1% relative input error** ($\delta x/x_0 = 0.01$) propagates to a $\approx \kappa \times 1\% =$ **50% relative output error**
- κ grows without bound as $|x_0| \to \infty$

Computer Arithmetic & Numerical Accuracy

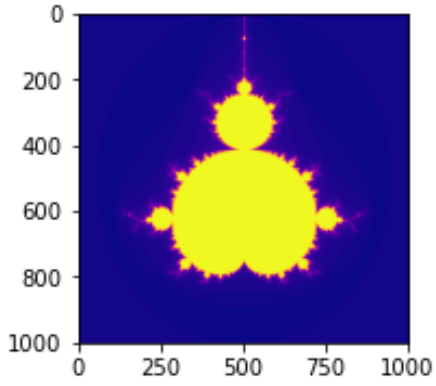
Condition Number — Mandelbrot Connection

$f(c) = n_{\text{escape}}(c)$: complex coord $\rightarrow$ escape iteration.

- **Exterior** (escapes quickly): low κ — float32 reliable
- **Interior** (never escapes): $\kappa = 0$ — float32 reliable
- **Boundary** (escapes near max_iter):
  - Chaotic: nearby points can diverge after many iterations
  - $\kappa(f, c) \rightarrow \infty$ at the fractal boundary
  - Rounding errors accumulate through hundreds of iterations $\Rightarrow$ wrong escape count in float32

Float32 is **unreliable exactly where computation is hardest.**

In L03 M4 you ran float32 and float64 at the default view and found them identical. Today you map where they are not.



Boundary regions: high κ, float32 unreliable

Computer Arithmetic & Numerical Accuracy

MP3 — Now Open

MP3 is open from today — due Sunday 27 April 23:59

Numerical experiments (today):

- M1: Trajectory divergence (float32 vs float64)
- M2: Sensitivity map (condition number approximation)

New implementations (L09–L10):

- GPU Mandelbrot (PyOpenCL)
- Unit tests with pytest
- Docstrings and a README

What to hand in:

- Code (.py or .ipynb) + test suite
- Performance report (all implementations)
- `git_log.txt`
- Reflection in Moodle text field (150+ words)

Computer Arithmetic & Numerical Accuracy

Studio Overview

Exercises (no hand-in):

- **E1**: Find machine epsilon experimentally
- **E2**: Catastrophic cancellation — quadratic formula (naive vs stable)
- **E3 (optional)**: Error accumulation in a loop

MP3 Milestones:

- **MP3 M1**: Mandelbrot trajectory divergence (float32 vs float64)
- **MP3 M2**: Mandelbrot sensitivity map

Studio Session

E1: Machine Epsilon

Instructions

Goal: find machine epsilon experimentally by successive halving, then compare to the standard value `np.finfo(dtype).eps`.

Algorithm:

1. Start with `eps = dtype(1.0)`
2. Repeat: `eps = eps / 2` until `dtype(1.0) + eps/2 == dtype(1.0)`
3. The last value where `1.0 + eps/2 != 1.0` is machine epsilon

Run for: `np.float16`, `np.float32`, `np.float64`

✓ **Done?** Do your computed values match `np.finfo`? → discuss with a neighbour.

E1: Machine Epsilon

Code example


```python
import numpy as np

def find_machine_epsilon(dtype=np.float64):
    eps = dtype(1.0)
    while dtype(1.0) + eps / dtype(2.0) != dtype(1.0):
        eps = eps / dtype(2.0)
    return eps

for dtype in [np.float16, np.float32, np.float64]:
    computed  = find_machine_epsilon(dtype)
    reference = np.finfo(dtype).eps
    print(f"{dtype.__name__}:")
    print(f"  Computed:  {float(computed):.4e}")
    print(f"  np.finfo:  {float(reference):.4e}")
    print()
```

E2: Catastrophic Cancellation

Instructions

Goal: observe precision loss when subtracting nearly-equal numbers, and fix it by reformulating the algorithm.

Test polynomial: $x^2 - 10000.0001\,x + 1 = 0$

- Roots: $x_1 \approx 10000.0001$ and $x_2 \approx 10^{-4}$
- In the naive formula, computing x_2 requires subtracting two large, nearly-equal numbers — losing all significant digits in float32

Tasks:

1. Implement the naive quadratic formula; run for float32 and float64
2. Implement the stable version using Vieta's formula ($x_1 \cdot x_2 = c/a$)
3. Compare relative error for the small root x_2 in both versions

NumPy dtype tip: In NumPy < 2.0, Python int literals, `b**2`, and `np.sqrt` silently upcast float32 to float64. Inside your function add `t = type(a)` and use `b*b`, `t(4)*a*c`, `t(2)*a`, `disc = t(np.sqrt(...))` to keep everything in the target dtype.

✓ **Done?** Record the relative error (naive vs stable, float32) → discuss with a neighbour.

E2: Catastrophic Cancellation

Code example — naive formula


```python
import numpy as np

def quadratic_naive(a, b, c):
    t = type(a)                          # np.float32 or np.float64
    disc = t(np.sqrt(b*b - t(4)*a*c)) # b*b not b**2; t() casts literals and sqrt
    x1 = (-b + disc) / (t(2)*a)
    x2 = (-b - disc) / (t(2)*a)
    return x1, x2
# x^2 - 10000.0001*x + 1 = 0    roots: x1 ~ 10000, x2 ~ 1e-4
for dtype in [np.float32, np.float64]:
    a, b, c = dtype(1.0), dtype(-10000.0001), dtype(1.0)
    x1, x2 = quadratic_naive(a, b, c)
    print(f"{dtype.__name__}: x1 = {float(x1):.4f},  x2 = {float(x2):.10f}")
```

E2: Catastrophic Cancellation

Code example — stable formula


```python
import numpy as np

def quadratic_stable(a, b, c):
    t = type(a)
    disc = t(np.sqrt(b*b - t(4)*a*c))
    if b > 0:
        x1 = (-b - disc) / (t(2)*a)  # pick sign that avoids cancellation
    else:
        x1 = (-b + disc) / (t(2)*a)
    x2 = c / (a * x1)   # Vieta's formula: x1 * x2 = c/a
    return x1, x2

true_small = 1.0 / 10000.0001   # ~ 1e-4
for dtype in [np.float32, np.float64]:
    a, b, c = dtype(1.0), dtype(-10000.0001), dtype(1.0)
    _, x2_naive  = quadratic_naive(a, b, c)
    _, x2_stable = quadratic_stable(a, b, c)
    err_naive  = abs(float(x2_naive)  - true_small) / true_small
    err_stable = abs(float(x2_stable) - true_small) / true_small
    print(f"{dtype.__name__}:  naive={err_naive:.2e}  stable={err_stable:.2e}")
```

E3 (Optional): Error Accumulation

Instructions

Task: add `0.1` in a loop n times; compare the result against the exact value $n \times 0.1$ for float32 and float64.

- Try $n \in \{10,\ 100,\ 1\,000,\ 10\,000,\ 100\,000\}$
- Compute relative error: `|result - expected| / expected`
- Does the error grow with n? Is float64 better?

Why does 0.1 cause problems? It has no exact binary representation — like $1/3$ in decimal, it repeats forever. Each addition compounds the truncation error.

Optional reading: Eijkhout §3.6.2 (Summing series) discusses related accumulation patterns and summation order strategies.

✓ **Done?** Observe how error grows with $n \rightarrow$ discuss with a neighbour

E3 (Optional): Error Accumulation

Code example


```python
import numpy as np

n_values = [10, 100, 1_000, 10_000, 100_000]

for dtype in [np.float32, np.float64]:
    print(f"\n{dtype.__name__}:")
    for n in n_values:
        total = dtype(0.0)
        for _ in range(n):
            total += dtype(0.1)
        expected  = n * 0.1
        rel_error = abs(float(total) - expected) / expected
        print(f"  n={n:>7d}:  result={float(total):.10f}  rel_error={rel_error:.2e}")
```

MP3 M1: Mandelbrot Trajectory Divergence

Instructions

Goal: find where float32 and float64 Mandelbrot trajectories diverge, and map the divergence iteration over the grid.

Region (default): Seahorse Valley — $x \in [-0.7530, -0.7490]$, $y \in [0.0990, 0.1030]$, `max_iter = 1000`

Algorithm:

1. Build two coordinate arrays: `C32 = C.astype(np.complex64)` and `C64`
2. Iterate $z_{n+1} = z_n^2 + c$ simultaneously for both dtypes, step by step
3. Record the first iteration k where $|z_{32}^{(k)} - z_{64}^{(k)}| > \tau$ (try $\tau = 0.01$)
4. Map these divergence iterations over the grid with `plt.imshow`

Observations to make:

- What fraction of pixels diverge before `max_iter`?
- Where do trajectories diverge *early*? Compare visually to the escape-count map.
- Does early divergence correlate with high escape iteration counts?

✓ **Done?** Record region, τ, and a brief observation in your performance notebook (MP3) → commit.

MP3 M1: Mandelbrot Trajectory Divergence

Code example


```
import numpy as np
import matplotlib.pyplot as plt
N, MAX_ITER, TAU = 512, 1000, 0.01
x = np.linspace(-0.7530, -0.7490, N)
y = np.linspace( 0.0990,  0.1030, N)
C64 = (x[np.newaxis, :] + 1j * y[:, np.newaxis]).astype(np.complex128)
C32 = C64.astype(np.complex64)
z32     = np.zeros_like(C32)
z64     = np.zeros_like(C64)
diverge = np.full((N, N), MAX_ITER, dtype=np.int32)
active  = np.ones((N, N), dtype=bool)
for k in range(MAX_ITER):
    if not active.any(): break
    z32[active] = z32[active]**2 + C32[active]
    z64[active] = z64[active]**2 + C64[active]
    diff = (np.abs(z32.real.astype(np.float64) - z64.real)
          + np.abs(z32.imag.astype(np.float64) - z64.imag))
    newly = active & (diff > TAU)
    diverge[newly] = k
    active[newly]  = False
plt.imshow(diverge, cmap='plasma', origin='lower',
           extent=[-0.7530, -0.7490, 0.0990, 0.1030])
plt.colorbar(label='First divergence iteration')
plt.title(f'Trajectory divergence (tau={TAU})')
plt.show()
```

MP3 M2: Mandelbrot Sensitivity Map

Instructions

Goal: compute a per-pixel condition number approximation for $f(c) = n_{\text{escape}}(c)$.

From theory: $\kappa(f, c) = \left| \frac{c\, f'(c)}{f(c)} \right|$. Substituting $f'(c) \approx \Delta n / \delta$ and $\delta = \varepsilon_{32} \cdot |c|$:

$$\kappa(c) \;\approx\; \frac{|\Delta n|}{\varepsilon_{32} \cdot n(c)}, \qquad \Delta n = n(c + \delta) - n(c)$$

Region: same as M1 (Seahorse Valley, `max_iter = 1000`)

Algorithm:

1. Compute $n(c)$ and $n(c + \delta)$; $\delta = \varepsilon_{32} \cdot |c|$
2. $\kappa(c) = |\Delta n| \,/\, (\varepsilon_{32} \cdot n(c))$; use `np.nan` where $n(c) = 0$
3. Plot with `cmap='hot'` and `LogNorm` (κ spans many orders of magnitude; linear scale → black image). Use `vmin=1` ($\kappa < 1$ is well-conditioned). Set `cmap.set_bad('0.25')` so NaN pixels (grey) are not confused with high-κ pixels (white).

Observations:

- Where is κ largest? Does it match the boundary in M1?
- What is κ for interior pixels ($n =$ `max_iter`)?

✓ **Done?** Add a brief note comparing M1 and M2 maps to your performance notebook (MP3) → commit.

MP3 M2: Mandelbrot Sensitivity Map

Code example


```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import LogNorm
N, MAX_ITER = 512, 1000
x = np.linspace(-0.7530, -0.7490, N)
y = np.linspace( 0.0990,  0.1030, N)
C = (x[np.newaxis, :] + 1j * y[:, np.newaxis]).astype(np.complex128)
eps32 = float(np.finfo(np.float32).eps)
delta = np.maximum(eps32 * np.abs(C), 1e-10)
def escape_count(C, max_iter):
    z = np.zeros_like(C); cnt = np.full(C.shape, max_iter, dtype=np.int32)
    esc = np.zeros(C.shape, dtype=bool)
    for k in range(max_iter):
        z[~esc] = z[~esc]**2 + C[~esc]
        newly = ~esc & (np.abs(z) > 2.0)
        cnt[newly] = k; esc[newly] = True
    return cnt
n_base    = escape_count(C,         MAX_ITER).astype(float)
n_perturb = escape_count(C + delta, MAX_ITER).astype(float)
dn = np.abs(n_base - n_perturb)
kappa = np.where(n_base > 0, dn / (eps32 * n_base), np.nan)
cmap_k = plt.cm.hot.copy(); cmap_k.set_bad('0.25')
vmax = np.nanpercentile(kappa, 99)
plt.imshow(kappa, cmap=cmap_k, origin='lower',
           extent=[-0.7530, -0.7490, 0.0990, 0.1030],
           norm=LogNorm(vmin=1, vmax=vmax))
plt.colorbar(label=r'$\kappa(c)$ (log scale, $\kappa \geq 1$)')
plt.title(r'Condition number approx  $\kappa(c) = |\Delta n|\,/\,(\varepsilon_{32}\,n(c))$')
plt.show()
```